Chapter 9 - Performance

Query Analysis

- There are many factors that can negatively affect the performance of a query, such as incorrect scaling, incorrectly structured collections, and inadequate resources such as RAM and CPU.

- However, the biggest and most common factor is the difference between the number of records scanned and the number of records returned during the query execution.

- The greater the difference is, the slower the query will be. Thankfully, in MongoDB, **this factor is the easiest to address and is done using indexes**.

- Creating and using indexes on a collection narrows down the number of records being scanned and improves the query performance noticeably.

Query Execution (My Section)

- To execute any such query, the MongoDB query execution engine prepares one or more query execution plans. The database has an inbuilt query optimizer that chooses the most efficient plan for the execution.

- A plan is usually composed of multiple processing stages that are executed in sequence to produce the final output.

- **The 4 query execution stages are:**
- **The Index Scan Stage**. Here, all available indexes are scanned to support the input query.
- **The Collection Scan Stage**: Here, the collection is scanned to find the docs for the input query.
- **The Sort Stage**: Here, the results are sorted as desired for the input query.
- **The Projection Stage**: Here, the project is applied as per the input query.

Explaining The Query

- The **explain()** function is useful in explaining the inner workings of a query. The function can be used along with a query or a command to print detailed statistics pertinent to their execution.

- The most important metrics that it can give us are: Query execution time, Number of documents scanned, Number of documents returned and the index that was used.

- By default, the **explain function prints the query planner details**—that is, details of various execution stages.

- The output shows the winning plan and a list of rejected plans.

- The execution stats provide useful metrics pertinent to each execution phase, along with some top-level fields where some metrics are aggregated over the total

execution of the query. The following are the most important metrics from the execution stats: **executionTimeMillis**, **totalKeysExamined**, **totalDocsExamined** and **nReturned**.

Linear Search

- When we execute a find query with a search criterion on a collection, the database search engine picks the first record in the collection and checks whether it matches the given criteria. If no match is found, the search engine moves on to the next record to find a match, and the process is repeated till a search is found.

- This search technique is called a sequential or linear search. Linear searches perform better when they are applied to a small amount of data, or in the best-case scenarios, where the required term is found within the first search.

- **Linear searches can be avoided or at least limited by creating indexes on specific fields of a collection. In the next section, we will explore this concept in detail.**

Introduction To Indexes

- Databases can maintain and use indexes to make searches more efficient. In MongoDB, indexes are created on a field or a combination of fields.

- The database maintains a special registry of indexed fields and some of their data. The registry is easily searchable, as it maintains a logical link between the value of an indexed field and the respective documents in the collection.

- During a search operation, the database first locates the value in the registry and identifies the matching documents in the collection accordingly. The values in a registry are always sorted in ascending or descending order of the values, which helps during a range search and also while sorting the results.

Creating And Listing Indexes

- Indexes can be created by executing a **createIndex() command** on a collection, as follows:

- The first argument to the command is a list of key-value pairs, where each pair consists of a field name and sort order, and the optional second argument is a set of options to control the indexes.

- We can create an index on the field / fields that we search regularly using the syntax **{ fieldname: value}** where **value** is 1 for ascending order and -1 for descending order.

- The output indicates that the index was successfully created. It also mentions the number of indexes present before and after the execution of this command and the time at which the index was created.

Listing Indexes On A Collection

- You can list the indexes of a collection by using the getIndexes() command.

- This command does not take any parameters. It simply returns an array of indexes with some basic details.

Exercise 9.01: Creating an Index Using MongoDB Atlas - Page 434

Hiding And Dropping Indexes

- Dropping an index means removing the values of the fields from the index registry. Thus, any searches on the related fields will be performed in a linear fashion, provided there are no other indexes present on the field.

- An incorrectly created index cannot be modified. To do that, we have to drop that index and recreate it.

- An index is deleted using the dropIndex function. It takes a single parameter, which can either be the index name or the index specification document. **The index specification document is the definition of the index that is used to create it**.

- The output contains nIndexesWas (highlighted), which refers to the index count before the command was executed. The ok field shows the status as 1, which indicates the command was successful.

- You can also drop multiple indexes using the **dropIndexes() command**. It can drop either a single index or multiple indexes. All the indexes except the default **_id** index can be dropped using this command. To drop multiple indexes, we need to pass an array of index names to this command.

- MongoDB provides a way to hide indexes from the query planner. Creating and deleting indexes are expensive operations in terms of time.

- To hide an index, the **hideIndex() command** can be used on the collection with the name or document specification of the index to be hidden.

- **An important thing to note is that hidden indexes appear only on the getIndexes() function call. They are updated after every write operation on the collection. However, the query planner won't see these indexes, and so they cannot be used for executing queries.**

Exercise 9.02: Dropping an Index Using Mongo Atlas - Page 444

Types of Indexes

- **Default Index:** As seen in the previous chapters, each document in a collection has a primary key (namely, the **_id field**) and is indexed by default. MongoDB uses this index to maintain the uniqueness of the **_id field**.

- **Single Key Index:** An index created using a single field from a collection is called a single-key index.

- **Compound Index:** A Compound Index is an index based on more than one field.

- **Multikey Index:** An index created on the fields of an array type is called a **multikey index**. When an array field is passed as an argument to the **createIndex function**, MongoDB creates an index entry for each element of the array.

- **Text Index:** An index defined on a string field or an array of string elements is called a text index. Text indexes are not sorted, meaning that they are faster than normal indexes.

Indexing Nested Fields

- Indexes can be created on sub-documents and also in fields nested within those sub-documents.

- MongoDB supports flexible schema, and different documents can have fields of varying types and quantities.

- When a new field is introduced into a document, it remains unindexed.

- **Wildcards** (Eg - *, $) can be used in indexes on fields.

- We can also select or omit specific fields from the wildcard indexes by passing a wildcardProjection option and one or more field names.

Properties of Indexes

- An index property can influence the usage of an index and can also enforce some behavior on the collection.

- A **unique index property** restricts the duplication of the index key. This is useful if you want to maintain the uniqueness of a field in a collection. The **{ unique: true }** option is used to create a unique index.

- To define a unique compound index, just pass the **{ unique: true }** option after defining all the other index fields.

- An index can't be created on a field in a collection if that field has duplicate values in the collection.

Exercise 9.03: Creating a Unique Index - Page 455

- **TTL (or Time to Live) indexes** put an expiry date on documents. Once the documents have expired, they are deleted. This index can only be created on a field of the date type. To create the index, you pass the field details and the **expireAfterSeconds** attribute.

- Here, the **{ expireAfterSeconds: seconds }** option is used to create a TTL index.

Exercise 9.04: Creating a TTL index using Mongo Shell - Page 458

Sparse Indexes

- When an index is created on a field, all the values of that field from all documents are maintained in the index registry. If the field does not exist in a document, a null value is registered for that document.

- **If an index is marked as sparse, then only those documents are registered in which the given field exists with some value including null. A sparse index will not have entries from the collection where the indexed field does not exist**.

- **Compound Indexes** can also be marked as sparse. For a compound sparse index, only those documents are registered where the combination of fields exists. Sparse indexes are created by passing a flag of **{ sparse: true }** to the **createIndex command**.

Exercise 9.05: Creating a Sparse Index Using Mongo Shell - Page 461

Partial Indexes

- An index can be created to maintain documents that match a given filter expression. Such an index is called a partial index. As the documents are filtered depending on the input expression, the size of the index is smaller than a normal index.

Exercise 9.06: Creating a Partial Index Using the Mongo Shell - Page 464

Case Insensitive Indexes

- Case-insensitive indexes allow you to find data using indexes in a case-insensitive manner. This means that the index will match the documents even if the values of a field are written in a different case from the values in the search expression.

Exercise 9.07: Creating a Case-Insensitive Index Using the Mongo Shell - Page 467

Other Query Optimization Techniques
Fetch Only What You Need

- **Correct Query Condition and Projection**: This can be done by using optimal query conditions and correctly using projections to return only the essential fields pertinent to the use case.

- **Pagination**: Pagination is about serving only a small subset of data to the client in each subsequent request.

Etc…